

МЕТОД РОЗВ'ЯЗАННЯ ЗАДАЧІ ПОШУКУ УСІХ НАЙБЛИЖЧИХ СУСІДІВ НА ОСНОВІ ТРИАНГУЛЯЦІЇ ДЕЛОНЕ

студент Тимофій ЛЕЙЦИШИН, група ТТП-32, 3 курс

Анотація. У роботі розглянуто задачу пошуку найближчого сусіда для кожної точки скінченної множини на евклідовій площині. Описано точний метод, який будує триангуляцію Делоне стратегією “розділай та пануй”, формує граф суміжності Делоне і для кожної точки переглядає лише суміжні вершини. Теоретична оцінка часу становить $\mathcal{O}(N \log N)$; практична частина містить програмну реалізацію геометричного ядра, покрокове відображення алгоритму та емпіричну перевірку на випадкових і структурованих наборах точок.

Abstract. The paper studies the all-nearest-neighbors problem for a finite point set in the Euclidean plane. The exact method described in the paper builds the Delaunay triangulation by a divide-and-conquer strategy, extracts the Delaunay adjacency graph, and checks only adjacent vertices for each site. The theoretical running time is $\mathcal{O}(N \log N)$; the practical part includes a self-contained geometry core, step-by-step algorithm visualization, and empirical evaluation on random and structured point sets.

1 Вступ

Постановка проблеми. Нехай задано множину $S = \{p_1, p_2, \dots, p_N\} \subset E^2$. Потрібно для кожної точки p_i знайти множину її найближчих сусідів

$$\text{NN}(p_i) = \left\{ p_j \in S \setminus \{p_i\} : \text{dist}(p_i, p_j) = \min_{p_k \in S \setminus \{p_i\}} \text{dist}(p_i, p_k) \right\}.$$

Якщо мінімальна відстань досягається на кількох точках, усі такі точки мають бути включені до відповіді. Тому результатом є не одна пара точок, а орієнтоване відношення $p_i \rightarrow p_j$, яке в загальному випадку не є симетричним.

Актуальність задачі визначається тим, що відношення найближчого сусіда є базовим елементом геометричного пошуку, кластеризації, побудови просторових графів, аналізу зон впливу, розміщення сервісних центрів, моделювання природних структур і задач інтерактивної графіки [1-4]. Прямий перебір усіх пар точок дає простий алгоритм часу $\mathcal{O}(N^2)$, але при $N = 10^4$ він потребує близько 10^8 порівнянь відстаней. Отже, для практичної реалізації потрібна структура, яка зменшує кількість кандидатів без втрати точності.

Аналіз існуючих підходів. Для задачі усіх найближчих сусідів розглядаються методи, які безпосередньо формують відповідь для кожної точки множини. Методи, призначені лише для пошуку однієї найближчої пари або для наближеного пошуку, не включаються до порівняння, оскільки вони розв'язують інші постановки. Порівняльну характеристику релевантних підходів наведено в таблиці 1.

Табл. 1: Порівняльна характеристика методів пошуку усіх найближчих сусідів.

Підхід	Основна ідея	Оцінка придатності для цієї роботи
Прямий перебір	Для кожної точки p_i обчислюються відстані до всіх $p_j \neq p_i$; відповідь формується без допоміжної геометричної структури.	Метод є точним і придатним для контрольної перевірки, але має часову складність $\mathcal{O}(N^2)$ і не відповідає заданій асимптотичній вимозі.
Просторова сітка	Площина розбивається на комірки; кандидати для точки шукаються в її комірці та в сусідніх комірках з можливим розширенням області пошуку.	Метод може мати малу сталу на рівномірних розподілах, але у гіршому випадку при нерівномірній концентрації точок зводиться до великої кількості попарних перевірок.
k -d-дерево	Будується дерево просторового розбиття, після чого для кожної точки виконується точний запит найближчого сусіда [5].	Метод застосовний для практичного пошуку, проте точний запит не має гарантованої логарифмічної межі для довільного розподілу точок; усі рівні мінімальні відстані також потребують окремої обробки.
Діаграма Вороного	Для кожної точки будується область близькості; найближчий сусід має спільну границю області Вороного з відповідною точкою [2, 4].	Структура дає точну відповідь за $\mathcal{O}(N \log N)$, але повна побудова та зберігання полігональних областей є надлишковими для задачі, де достатньо графа суміжності.
Триангуляція Делоне	Використовується граф, дуальний до діаграми Вороного; найближчий сусід кожної точки належить множині суміжних вершин у графі Делоне [3, 4, 6, 7].	Метод, використаний у роботі. Він зберігає всіх необхідних кандидатів у планарному графі лінійного розміру та сумісний з рекурсивною побудовою за $\mathcal{O}(N \log N)$.

За наведеним порівнянням, для точної задачі на площині з вимогою $\mathcal{O}(N \log N)$ достатньою структурою є граф суміжності Делоне. Повна геометрія областей Вороного не використовується, оскільки для формування відповіді потрібні лише суміжні точки, а не полігональна форма кожної області.

Науково-практична новизна. У межах цієї роботи вона полягає у побудові самодостатнього програмного методу для точного розв'язання задачі усіх найближчих сусідів на площині на основі графа Делоне. Метод не змінює відому асимптотичну межу і не формулює нового твердження про діаграму Вороного, однак поєднує в одній реалізації побудову триангуляції Делоне без готового алгоритмічного пакета, перехід до графа кандидатів, збереження всіх рівновіддалених мінімальних сусідів, спеціальну обробку колінеарного випадку та покрокове відображення інваріантів алгоритму. Така комбінація має прикладне значення, оскільки забезпечує не лише отримання відповіді, а й контроль проміжних геометричних операцій.

Мета роботи. Розробити, обґрунтувати і реалізувати ефективний метод знаходження усіх найближчих сусідів для множини точок на площині за час $\mathcal{O}(N \log N)$, забезпечивши ручне та автоматичне введення даних, покрокове відображення алгоритму, перевірку коректності на малих наборах і емпіричну оцінку ефективності на великих наборах точок.

2 Основна частина

2.1. Постановка задачі

Нехай задано множину S з N різних скінченних точок на площині:

$$S = \{p_i = (x_i, y_i) : i = 1, \dots, N\} \subset \mathbb{R}^2.$$

Потрібно для кожного $p_i \in S$ знайти усіх найближчих сусідів $\text{NN}(p_i)$. Вхідними даними є координати точок. Вихідними даними є список орієнтованих пар $p_i \rightarrow p_j$ та значення квадрату мінімальної відстані для кожної точки. Вимога до алгоритму - час $\mathcal{O}(N \log N)$ для множини з N точок.

У реалізації вважаються некоректними точні дублікати координат, нескінченні координати та набори з менш ніж двох різних точок. Колінеарний випадок обробляється як спеціальний одновимірний випадок: точки впорядковуються, після чого достатньо перевірити сусідів у відсортованому ланцюзі.

2.2. Основні означення

Означення 1. Відстанню між двома точками $a = (x_a, y_a)$ і $b = (x_b, y_b)$ на площині називається величина

$$\text{dist}(a, b) = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}.$$

У реалізації для порівнянь використовується квадрат відстані

$$\text{dist}^2(a, b) = (x_a - x_b)^2 + (y_a - y_b)^2,$$

оскільки квадратний корінь не змінює порядку відстаней.

Означення 2. Точка $b \in S \setminus \{a\}$ називається найближчим сусідом точки $a \in S$, якщо для кожної точки $c \in S \setminus \{a\}$ виконується нерівність $\text{dist}(a, b) \leq \text{dist}(a, c)$.

Означення 3. Областю Вороного точки $p_i \in S$ називається множина

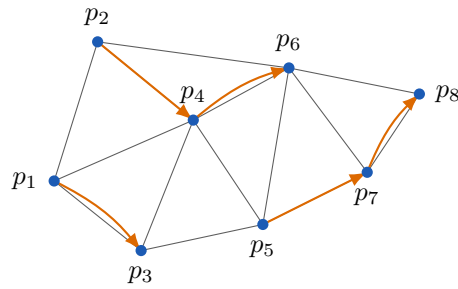
$$V_i = \{x \in \mathbb{R}^2 : \text{dist}(x, p_i) \leq \text{dist}(x, p_j) \text{ для всіх } p_j \in S\}.$$

Сукупність областей V_i утворює діаграму Вороного $\text{Vor}(S)$ [2, 4].

Означення 4. Триангуляцією Делоне $\text{DT}(S)$ називається планарний граф, дуальний до діаграми Вороного. У ньому дві точки p_i і p_j з'єднані ребром, якщо їхні області Вороного мають спільну границю або, еквівалентно для не вироджених випадків, існує порожнє коло, що проходить через p_i і p_j .

2.3. Теоретичне обґрунтування методу

Лема 1. Якщо точка q є найближчим сусідом точки p у множині S , то ребро pq належить графу Делоне або граничному варіанту Делоне для виродженої конфігурації з рівними відстанями.



сірі ребра - граф Делоне; помаранчеві стрілки - вибрані найближчі сусіди

Рис. 1: Зведення глобального пошуку до локального перегляду графа Делоне.

Доведення. Нехай $d = \text{dist}(p, q)$. Оскільки q є найближчим сусідом p , для кожної точки $r \in S \setminus \{p, q\}$ маємо $\text{dist}(p, r) \geq d$. Розглянемо коло з діаметром pq . Кожна внутрішня точка цього кола має відстань до p , строго меншу за d , отже жодна інша точка множини S не може лежати всередині цього кола. Тому існує порожнє коло, що проходить через p і q . За критерієм Делоне пара p, q визначає ребро графа Делоне; у випадку співковості воно належить до допустимого граничного графа або до однієї з допустимих діагоналізацій. Отже, найближчий сусід завжди міститься серед Делоне-суміжних кандидатів. $\square$

Твердження 1. Для знаходження усіх найближчих сусідів достатньо побудувати триангуляцію Делоне і для кожної вершини переглянути лише її суміжні вершини.

Доведення. З леми випливає, що кожен істинний найближчий сусід точки p_i міститься в множині $\text{Adj}_{\text{DT}}(p_i)$. Тому локальний мінімум за списком суміжності збігається з глобальним мінімумом за всією множиною:

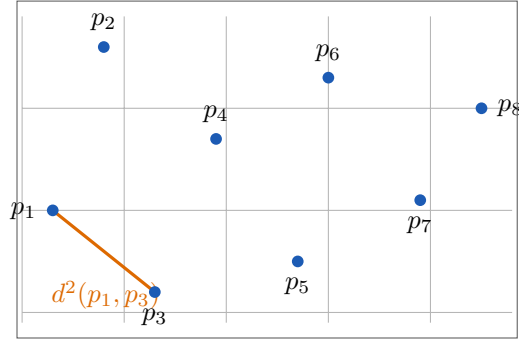
$$\min_{q \in S \setminus \{p_i\}} \text{dist}^2(p_i, q) = \min_{q \in \text{Adj}_{\text{DT}}(p_i)} \text{dist}^2(p_i, q).$$

Якщо існує кілька точок на тій самій мінімальній відстані, усі вони проходять це саме порівняння і додаються до результату. $\square$

2.4. Запропонований алгоритм і покрокова візуалізація

Метод складається з двох логічно пов'язаних частин. На першому етапі для множини точок будується триангуляція Делоне методом рекурсивного поділу та злиття. На другому етапі з отриманої триангуляції будується граф суміжності, після чого для кожної вершини переглядаються лише її сусіди у графі Делоне.

Крок 1. Перевірка вхідних даних та вибір метрики

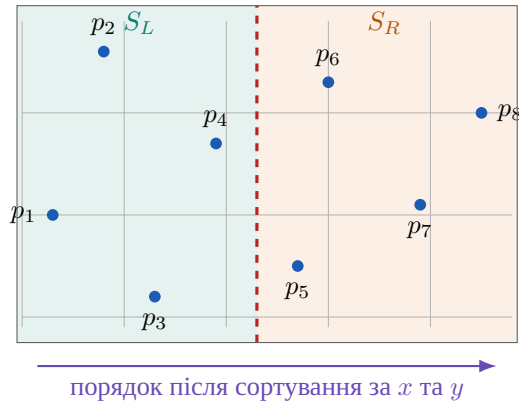


$$S = \{p_i = (x_i, y_i)\}_{i=1}^N, \quad N \geq 2, \quad p_i \neq p_j \ (i \neq j), \quad x_i, y_i \in \mathbb{R},$$

$$d^2(p_i, p_j) = (x_i - x_j)^2 + (y_i - y_j)^2.$$

Рис. 2: Вхідна множина точок та квадрат евклідової відстані, який використовується для порівняння кандидатів.

Крок 2. Впорядкування точок та початковий поділ множини

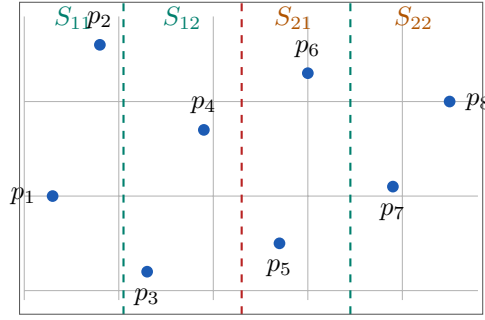


$$(p_{\pi(1)}, \dots, p_{\pi(N)}) = \text{sort}_{x,y}(S), \quad S_L = \{p_{\pi(1)}, \dots, p_{\pi(m)}\}, \quad S_R = \{p_{\pi(m+1)}, \dots, p_{\pi(N)}\},$$

$$m = \left\lfloor \frac{N}{2} \right\rfloor, \quad ||S_L| - |S_R|| \leq 1.$$

Рис. 3: Сортування множини та перший рекурсивний поділ відсортованого списку за медіаною.

Крок 3. Рекурсивний поділ до базових підзадач

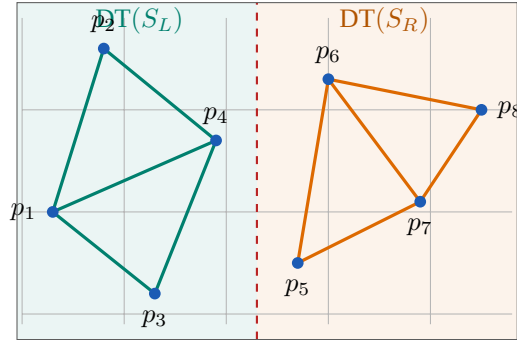


$$DT(S) = \text{merge}(DT(S_L), DT(S_R)), \quad DT(S_L) = \text{merge}(DT(S_{11}), DT(S_{12})),$$

$$DT(S_R) = \text{merge}(DT(S_{21}), DT(S_{22})).$$

Рис. 4: Рекурсивне розбиття відсортованої множини до базових підмножин малого розміру.

Крок 4. Побудова часткових триангуляцій у підзадачах

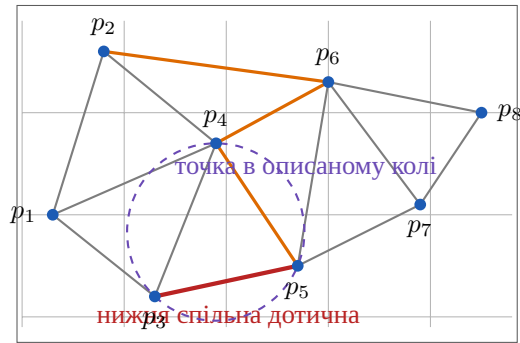


$$|S'| = 2 \Rightarrow E' = \{\{a, b\}\}.$$

$$|S'| = 3 \Rightarrow \begin{cases} \text{утворюється трикутник,} & \text{orient}(a, b, c) \neq 0, \\ \text{утворюється впорядкований ланцюг,} & \text{orient}(a, b, c) = 0. \end{cases}$$

Рис. 5: Після розв'язання базових підзадач формуються дві часткові триангуляції, які ще не з'єднані між собою.

Крок 5. Злиття часткових триангуляцій Делоне

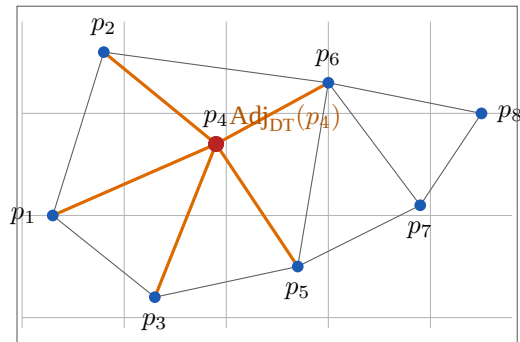


злиття починається з нижньої спільної дотичної;
далі послідовно перевіряються кандидати зліва і справа.

$d \in \text{int } C(a, b, c) \Rightarrow$ відповідне ребро порушує умову Делоне та вилучається.

Рис. 6: Злиття виконується додаванням міжчастинних ребер із контролем предикатом «точка в описаному колі».

Крок 6. Побудова графа суміжності Делоне

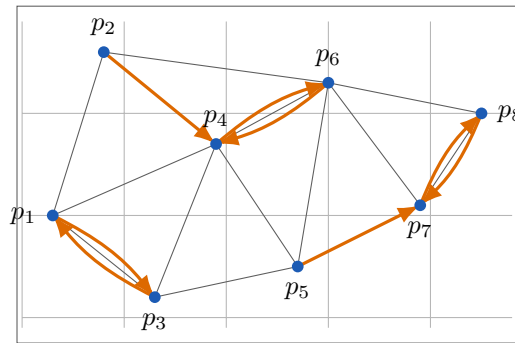


$$G_D = (S, E_D), \quad E_D = \{\{p_i, p_j\} : p_i p_j \in \text{DT}(S)\},$$

$$\text{Adj}_{\text{DT}}(p_i) = \{p_j : \{p_i, p_j\} \in E_D\}.$$

Рис. 7: З ребер триангуляції формується неорієнтований граф суміжності Делоне.

Крок 7. Вибір усіх найближчих сусідів



$$D_i = \min_{q \in \text{Adj}_{\text{DT}}(p_i)} d^2(p_i, q), \quad \text{NN}(p_i) = \{q \in \text{Adj}_{\text{DT}}(p_i) : d^2(p_i, q) = D_i\}.$$

Рис. 8: Для кожної вершини переглядаються лише її сусіди у графі Делоне, після чого зберігаються всі мінімальні за відстанню кандидати.

Відповідно до рисунків 2–8, алгоритм виконує такі етапи:

1. Перевірити коректність вхідних даних: координати мають бути скінченними, точні дублювати координат не допускаються, кількість різних точок має бути не меншою за дві.
2. Упорядкувати точки за координатою x , а при рівності x — за координатою y .
3. Якщо всі точки колінеарні, побудувати впорядкований ланцюг за відсортованим списком і перейти до побудови графа суміжності.
4. Рекурсивно розбити відсортований список на дві приблизно рівнопотужні частини за медіаною, а далі повторювати поділ до досягнення базових підзадач.
5. Для підмножин з двох або трьох точок побудувати базові ребра чи базовий трикутник залежно від значення предиката орієнтації; з них рекурсивно утворити часткові триангуляції лівої та правої частин.
6. Злити дві часткові триангуляції: знайти нижню спільну дотичну, додати базове ребро, послідовно перевіряти лівих і правих кандидатів, а ребра, що порушують умову Делоне, вилучати за допомогою предиката «точка в описаному колі».
7. Витягнути неорієнтовані ребра триангуляції та побудувати граф суміжності.
8. Для кожної точки переглянути її сусідів у графі Делоне, обчислити квадрати відстаней, вибрати мінімум та зберегти всі рівновіддалені мінімальні точки.

2.5. Основні геометричні предикати

У реалізації використовуються три предикати: квадрат відстані, орієнтація трійки точок та перевірка належності точки внутрішності описаного кола. Предикат орієнтації визначається знаком детермінанта

$$\text{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

Додатний знак відповідає лівому повороту, від'ємний - правому, нульовий - колінеарності.

Для кроку злиття триангуляції Делоне використовується предикат «точка в описаному колі». Нехай d - тестова точка, а a, b, c задають орієнтований трикутник. Тоді обчислюється знак детермінанта

$$\begin{vmatrix} a_x - d_x & a_y - d_y & (a_x - d_x)^2 + (a_y - d_y)^2 \\ b_x - d_x & b_y - d_y & (b_x - d_x)^2 + (b_y - d_y)^2 \\ c_x - d_x & c_y - d_y & (c_x - d_x)^2 + (c_y - d_y)^2 \end{vmatrix}.$$

Якщо знак вказує, що d лежить всередині описаного кола, поточний кандидат порушує умову Делоне, і відповідне ребро вилучається. Через використання чисел з плаваючою комою порівняння виконується з малим порогом $\varepsilon = 10^{-9}$; для реалізацій із гарантованою числовою стійкістю застосовують адаптивні точні предикати [8].

2.6. Псевдокод

Listing 1: Скорочений псевдокод методу усіх найближчих сусідів.

```

1 AllNearestNeighbors(S):
2   P ← normalize(S)
3   P ← sortByXThenY(P)
4   if P is collinear:
5     E ← sortedChainEdges(P)
6   else:
7     E ← DivideConquerDelaunay(P)
8   G ← adjacencyGraph(E)
9   for each point p in P:
10    best ← infinity
11    NN[p] ← empty list
12    for each q in G.adjacent(p):
13      d2 ← squaredDistance(p, q)
14      if d2 < best:
15        best ← d2
16        NN[p] ← [q]
17      else if d2 = best:
18        append q to NN[p]
19  return NN

```

3 Обґрунтування складності

Теорема 1. Для N різних точок на площині запропонований алгоритм знаходить усіх найближчих сусідів за час $\mathcal{O}(N \log N)$ і використовує $\mathcal{O}(N)$ додаткової пам'яті після зберігання вхідних точок.

Доведення. Розглянемо всі етапи алгоритму окремо.

Перевірка вхідних даних виконує один перегляд вхідного масиву і пошук дублікатів у хеш-таблиці. Її очікуваний час дорівнює $\mathcal{O}(N)$, а пам'ять - $\mathcal{O}(N)$. Сортування точок за x та y потребує $\mathcal{O}(N \log N)$ операцій порівняння.

Побудова триангуляції Делоне методом “розділяй та пануй” має рекурентну оцінку

$$T(N) = 2T(N/2) + M(N),$$

де $M(N)$ - час злиття двох триангуляцій. У merge-кроці знаходиться спільна дотична, а потім будується ланцюг базових ребер. Кожна перевірка кандидата або просуває ланцюг, або вилучає ребро, яке більше не може належати злитій триангуляції. Оскільки триангуляція Делоне є планарним графом, кількість її ребер лінійна: для $N \geq 3$ маємо не більше $3N - 6 - h$ ребер, де h - кількість точок на опуклій оболонці. Тому сумарна кількість просувань і вилучень

на одному рівні злиття є лінійною, тобто $M(N) = \mathcal{O}(N)$. За теоремою Майстра отримуємо $T(N) = \mathcal{O}(N \log N)$.

Після побудови триангуляції витягнення ребер і формування графа суміжності потребує часу, пропорційного кількості ребер, тобто $\mathcal{O}(N)$. Далі кожна точка переглядає лише суміжні вершини. Сумарна кількість переглядів дорівнює подвоєній кількості ребер графа Делоне, тобто також $\mathcal{O}(N)$. Вибір мінімальних квадратів відстаней не збільшує оцінки.

Таким чином, домінуючим етапом є сортування і рекурсивна побудова триангуляції Делоне: загальний час $\mathcal{O}(N \log N)$. Структура quad-edge, список ребер, граф суміжності та список відповідей мають лінійний розмір, тому додаткова пам'ять становить $\mathcal{O}(N)$. $\square$

Порівняння з прямим методом. Прямий метод для кожного p_i перевіряє всі $N - 1$ інших точок, тому виконує $N(N - 1)$ порівнянь відстаней і має складність $\mathcal{O}(N^2)$. Запропонований алгоритм переносить основну роботу у побудову планарної структури $DT(S)$, після чого кількість кандидатів для всіх точок є лінійною. Саме ця редукція забезпечує виграш на великих N .

4 Практична частина

4.1. Особливості програмної реалізації

Програмна реалізація виконана як веб-застосунок на React та TypeScript. Геометричне ядро розміщене у модулях `src/geometry/*` і не залежить від React або D3. Бібліотека D3 використовується лише для SVG-відображення, зміни масштабу, переміщення області перегляду та показу результатів. Такий поділ дає змогу перевіряти алгоритмічну частину окремо від інтерфейсу.

Основні модулі реалізації:

- `delaunay-triangulation.ts` - рекурсивна побудова триангуляції Делоне та крок злиття;
- `quad-edge.ts` - топологічні операції quad-edge: `makeEdge`, `splice`, `connect`, `deleteEdge`;
- `predicates.ts` - квадрат відстані, орієнтація, предикат «точка в описаному колі», колінеарність;
- `adjacency-graph.ts` - перетворення списку ребер у граф суміжності;
- `nearest-neighbors.ts` - локальний пошук усіх найближчих сусідів за графом Делоне;
- `run-geometry-core.ts` - композиція геометричного ядра;
- `bruteForceNearest.ts` - контрольна перевірка малих наборів прямим методом;
- модулі покрокового відображення - перетворення подій алгоритму у кадри для покрокового відображення.

4.2. Опис основних функцій

Функція `runGeometryCore(points)` є основною функцією запуску алгоритмічної частини. Вона будує триангуляцію Делоне, отримує граф суміжності та повертає масив найближчих сусідів. Функція `delaunayTriangulation(points)` перевіряє дублікати, сортує точки за координатами, обробляє колінеарний випадок і запускає рекурсивне ділення. Функція

`nearestNeighbors(points, graph)` для кожної вершини переглядає лише суміжні вершини та зберігає всі рівні мінімальні відстані.

У коді навмисно використовується квадрат евклідової відстані. Це зменшує кількість дорогих операцій `sqrt`, не змінюючи результату порівняння, оскільки функція $x \mapsto \sqrt{x}$ є монотонно зростаючою для $x \geq 0$.

4.3. Характеризація введення-виведення даних

У програмі передбачено кілька режимів введення. Для демонстрації малих наборів користувач може додавати точки вручну у робочій області. Для перевірки ефективності передбачена автоматична генерація випадкових точок та структурованої решітки. Також підтримується завантаження точок з текстових, CSV або JSON-файлів. Користувач задає кількість точок перед генерацією.

Вихідні дані подаються у двох формах: графічно та таблично. У робочій області відображаються ребра Делоне, вибрані стрілки найближчих сусідів, проміжні кроки побудови, кандидати кроку злиття, описані кола та вилучені ребра. У панелі результатів подається список відповідностей $p_i \rightarrow p_j$ і службова інформація про перевірку коректності.

4.4. Інтерфейс і режими покрокового відображення

Інтерфейс містить робочу область, засоби зміни масштабу та переміщення області перегляду, команди додавання точок, очищення, запуску алгоритму, генерації, завантаження даних і масштабування до наявної множини точок. Передбачено також перемикачі візуальних шарів. Для $N \leq 100$ доступне детальне покрокове відображення, для $100 < N \leq 2000$ - відображення основних фаз, а для $N > 2000$ покрокове відображення вимикається, щоб зберегти прийнятний час реакції інтерфейсу на великих наборах. Такий режим відокремлює демонстрацію математичної ідеї на малих даних від вимірювання ефективності геометричного ядра на великих даних.

4.5. Програмне та технічне забезпечення

Для запуску потрібні Node.js, npm і сучасний браузер. Основні команди:

Listing 2: Команди запуску, тестування та вимірювання продуктивності.

```
1 npm install
2 npm run dev
3 npm test
4 npm run build
5 npm run benchmark
6 npm run benchmark:large
```

Реалізація не використовує готові бібліотеки `d3-delaunay`, `delaunator`, KD-tree або інші геометричні алгоритмічні пакети для побудови результату. D3 використовується лише для графічного відображення та взаємодії з SVG.

4.6. Емпіричні результати

Вимірювання охоплює лише виконання `runGeometryCore(points)`:

`delaunayTriangulation + adjacencyGraph + nearestNeighbors`.

Із часу виконання були виключені генерація даних, відображення React/D3, покрокове відображення, перевірка коректності прямим методом і запис файлів. Отже, наведені значення характеризують саме геометричне ядро.

Табл. 2: Фрагмент результатів вимірювання продуктивності для $N = 10000$.

Сімейство	N	Медіана, мс	Ребра	Зв'язки NN	$T/(N \log_2 N)$, мкс
random	10000	51.079	29974	10000	0.384
structured	10000	45.955	29940	10000	0.346

Для великих запусків надано окремі вимірювання до 10^6 точок. Вони не є математичним доведенням складності, однак узгоджуються з тим, що нормована величина $T(N)/(N \log_2 N)$ не демонструє квадратичного росту на протестованих наборах.

Табл. 3: Великі запуски геометричного ядра.

Сімейство	N	Медіана, мс	Ребра	Зв'язки NN	$T/(N \log_2 N)$, мкс
random	100000	780.794	299965	100000	0.470
random	1000000	12371.863	2999953	1000002	0.621
structured	100000	525.561	299857	100000	0.316
structured	1000000	7926.911	2999502	1000000	0.398

4.7. Перевірка коректності

Для малих наборів, де $N \leq 100$, реалізація порівнює відповідь алгоритму Делоне з відповіддю прямого алгоритму $\mathcal{O}(N^2)$. Такий контроль не замінює доведення, але дає змогу виявляти реалізаційні помилки у випадках з двома точками, трьома точками, колінеарністю, майже колінеарністю, випадковими наборами та рівними мінімальними відстанями. Для більших наборів повна перевірка прямим методом не виконується, оскільки вона суперечила б меті оцінювання ефективного алгоритму.

5 Висновки

У роботі розглянуто задачу пошуку усіх найближчих сусідів для множини точок на площині. Проаналізовано прямий перебір, сіткові методи, точні запити в k -d-дереві, діаграму Вороного та триангуляцію Делоне. Для точної задачі з вимогою $\mathcal{O}(N \log N)$ обрано підхід через триангуляцію Делоне, оскільки він зберігає всіх необхідних кандидатів у планарному графі лінійного розміру.

Основним практичним результатом є реалізація методу “Делоне $\rightarrow$ граф суміжності $\rightarrow$ усі найближчі сусіди” з підтримкою рівновіддалених мінімальних сусідів, спеціальною обробкою колінеарного випадку, покроковим відображенням кроків алгоритму і незалежною перевіркою малих наборів прямим методом. Теоретично доведено, що після сортування та побудови Делоне за стратегією “розділяй та пануй” сумарний перегляд кандидатів є лінійним, а загальна часова складність дорівнює $\mathcal{O}(N \log N)$.

Серед обмежень слід зазначити використання чисел з плаваючою комою для геометричних предикатів [8]. Це є стандартним практичним компромісом, однак сильно вироджені конфігурації можуть вимагати адаптивних точних предикатів. Подальший розвиток доцільно спрямувати на інтеграцію адаптивних точних предикатів орієнтації та перевірки точки в описаному колі, розширення системи контрольних прикладів для співколових конфігурацій, вимірювання використання пам'яті на дуже великих наборах, а також додавання експорту результатів у стандартні формати.

Список літератури

1. Терещенко В. М. Задачі близькості. Метод пошуку найближчої пари. Лекційні матеріали, 2025.
2. Терещенко В. М. Методи розв’язування задач на близькість. Діаграма Вороного. Лекційні матеріали, 2025.
3. Preparata F. P., Shamos M. I. Computational Geometry: An Introduction. Springer-Verlag, New York, 1985. 398 p.
4. de Berg M., van Kreveld M., Overmars M., Schwarzkopf O. Computational Geometry: Algorithms and Applications. 3rd ed. Springer-Verlag, Berlin Heidelberg, 2008. 386 p.
5. Bentley J. L. Multidimensional binary search trees used for associative searching. Communications of the ACM, Vol. 18, No. 9, 1975, pp. 509-517.
6. Lee D. T., Schachter B. J. Two algorithms for constructing a Delaunay triangulation. International Journal of Computer & Information Sciences, Vol. 9, No. 3, 1980, pp. 219-242.
7. Guibas L., Stolfi J. Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams. ACM Transactions on Graphics, Vol. 4, No. 2, 1985, pp. 74-123.
8. Shewchuk J. R. Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates. Discrete & Computational Geometry, Vol. 18, No. 3, 1997, pp. 305-363.

А Додатки

А.1. Фрагмент геометричного ядра

Listing 3: Композиція геометричного ядра.

```
1 export function runGeometryCore(points, trace) {
2   const triangulation = delaunayTriangulation(points, trace);
3   const graph = adjacencyGraph(points.length, triangulation.edges);
4   const nearest = nearestNeighbors(points, graph, trace);
5
6   return {
7     triangulation,
8     graph,
9     nearestNeighbors: nearest,
10  };
11 }
```

А.2. Фрагмент вибору найближчих сусідів

Listing 4: Локальний перегляд суміжності Делоне.

```
1 for (let index = 0; index < points.length; index += 1) {
2   const source = points[index];
3   const adjacent = graph.get(index) ?? new Set();
4   let bestDistance = Number.POSITIVE_INFINITY;
5   let neighbors = [];
6
7   for (const neighbor of adjacent) {
8     const target = points[neighbor];
```

```

9   const candidateDistance = distanceSquared(source, target);
10
11   if (candidateDistance + 1e-9 < bestDistance) {
12     bestDistance = candidateDistance;
13     neighbors = [neighbor];
14   } else if (sameDistance(candidateDistance, bestDistance)) {
15     neighbors.push(neighbor);
16   }
17 }
18
19 result.push({
20   point: index,
21   neighbors: neighbors.sort((a, b) => a - b),
22   distanceSquared: bestDistance,
23 });
24 }

```

A.3. Фрагмент геометричних предикатів

Listing 5: Орієнтація, квадрат відстані та предикат «точка в описаному колі».

```

1   export const EPSILON = 1e-9;
2
3   export function distanceSquared(a, b) {
4     const dx = a.x - b.x;
5     const dy = a.y - b.y;
6     return dx * dx + dy * dy;
7   }
8
9   export function orientation(a, b, c) {
10    return (b.x - a.x) * (c.y - a.y) -
11           (b.y - a.y) * (c.x - a.x);
12  }
13
14  export function inCircle(a, b, c, d) {
15    const ax = a.x - d.x;
16    const ay = a.y - d.y;
17    const bx = b.x - d.x;
18    const by = b.y - d.y;
19    const cx = c.x - d.x;
20    const cy = c.y - d.y;
21
22    const det =
23      (ax * ax + ay * ay) * (bx * cy - by * cx) -
24      (bx * bx + by * by) * (ax * cy - ay * cx) +
25      (cx * cx + cy * cy) * (ax * by - ay * bx);
26
27    return orientation(a, b, c) >= 0 ? det : -det;
28  }

```

A.4. Контрольні приклади, передбачені у реалізації

У тестових модулях передбачено перевірку квадрату відстані, орієнтації, предикат «точка в описаному колі», відхилення дублікатів, прямого алгоритму найближчих сусідів, методу на основі Делоне на малих наборах, покрокового відображення та колінеарних спеціальних випадків. Для малих наборів результат методу на основі Делоне порівнюється з прямим методом, що має незалежну реалізацію.